
$\mathcal{P}$ Torch: Narrowing the Gap Between Projection and Gradient-Based Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Projection-based optimization offers a gradient-free alternative to conventional
2 gradient based methods, but existing frameworks suffer from high computational
3 costs and poor scaling with depth. We introduce $\mathcal{P}$ Torch, a projection-based
4 framework that builds directly on top of PyTorch’s autograd engine, eliminating the
5 overhead of prior approaches. We further improve performance through nonlinear
6 relaxation and memory-efficient projections. Together, these contributions make the
7 algorithm competitive with gradient-based methods for shallow MLPs on MNIST
8 and CIFAR-10. Finally, we prove a vanishing target theorem showing that the
9 layer-wise learning signal decays with increased depth, serving as an analogue of
10 the vanishing gradients problem for projection-based methods.

11 1 Introduction

12 Gradient-based optimization lies at the heart of modern deep learning, but it rests on two structural
13 assumptions: every operation in the computational graph must admit a gradient (or a tractable
14 subgradient), and parameter updates must propagate through the chain rule. These assumptions have
15 motivated a long line of work on alternatives to backpropagation, ranging from classical proposals
16 such as target propagation [12] to more recent schemes such as forward-forward training [10] and
17 neuroevolution [16].

18 A complementary line of work recasts learning as a feasibility problem: rather than minimizing the
19 loss, one seeks a point that simultaneously satisfies a collection of constraints describing the network,
20 the data, and the parameters [2, 5, 6]. The appeal of this formulation is that each layer needs only
21 to admit a tractable projection, not a gradient. Because the full constraint set induced by a neural
22 network is far too complex to project onto directly, it is decomposed into subsets that individually
23 admit closed-form or computationally tractable projections. Standard projection algorithms can
24 then operate on this decomposition: cyclic projections [8] produce a feasible point, while Dykstra’s
25 algorithm [4] recovers the projection onto the intersection itself. The two-set case is particularly well
26 understood; in particular, Douglas-Rachford splitting has been studied extensively and applied across
27 a wide range of feasibility and optimization problems [1, 13].

$$\min_{\theta} \mathcal{L}(f(x; \theta), y) \implies \theta \in \bigcap_{i=1}^m C_i.$$

28
29 A caveat is that the convergence theory for these methods has been developed primarily in the convex
30 setting, whereas the constraint sets induced by neural networks are highly non-convex.

31 Recent work by Bergmeister [2] and Elser [5] has shown that projection-based algorithms can train
32 neural networks in practice, but only at the cost of substantial memory overhead and empirical

performance that remains well below that of gradient-based training. A closer examination reveals three recurring bottlenecks. First, memory consumption is severe: the consensus step in the linear layer scales with $O(N_{\text{in}}N_{\text{out}}B)$ rendering training infeasible on commonly available hardware. Second, the graph-propagation scheme of [2] requires splitting the computational graph into a bipartite graph for every batch, introducing a per-batch overhead absent from standard training pipelines. Third, convergence is noticeably slower than Adam, even on small MLPs. Moreover, these frameworks were developed largely in isolation from modern deep learning toolchains and therefore cannot benefit from the many software optimizations that underpin state-of-the-art libraries such as Torch [15].¹

Can we create a framework that trains neural networks with projections as effectively as PyTorch with gradients?

To address these issues, we developed $\mathcal{P}$ Torch. The compatibility problem is resolved by building on PyTorch rather than JAX, and by performing graph traversal directly through the native autograd. As a result, the gap between gradient-based and projection-based optimization in our framework reduces, as closely as possible, to the gap between backpropagation and its projection-based counterpart isolating the optimization strategy from implementation artifacts.

A further motivation for this line of research is the risk of paradigm lock-in: the overwhelming focus on gradient-based training may lead the community to overlook alternative learning frameworks that, given comparable research investment, could ultimately prove competitive or superior. Projection-based learning is still in its infancy, yet $\mathcal{P}$ Torch already achieves competitive performance on very shallow MLPs, suggesting that much of the gap was attributable to implementation artifacts rather than a fundamental limitation of the optimization strategy.

Making projection-based learning a mainstream paradigm remains a long-term goal; as with gradient-based optimization, the right implementation tricks to train deep networks such as weight-initialization like He initialization [9] for gradients have yet to be fully worked out.

Contributions. We make the following contributions:

1. **A cyclic projection algorithm for neural networks.** We establish a conceptual connection between cyclic projections, the projection-based learning algorithms of [5, 2], and PyTorch’s autograd graph traversal. Leveraging this connection, we develop a variant that converges faster both per step and in wall-clock time than existing projection-based methods, while remaining competitive with standard PyTorch optimizers on shallow networks.
2. **Efficient ℓ_2 and ℓ_∞ linear layers.** For the ℓ_2 norm, we derive a consensus formula that reduces memory complexity from $O(N_{\text{in}}N_{\text{out}}B)$ to $O(N_{\text{in}}B + N_{\text{out}}B + N_{\text{in}}N_{\text{out}})$. For the ℓ_∞ norm, we derive a variant that can be solved in $O(n)$ time.
3. **First-order optimizers as nonlinear relaxation.** By making the connection between incremental gradient descent and the cyclic projection update rule, we show that first-order optimizers such as Adam and Muon can be naturally incorporated as nonlinear relaxation methods, yielding consistent and substantial empirical gains.
4. **Hybrid gradient- and projection-based training.** Our framework integrates with PyTorch, enabling users to mix projection and gradient steps arbitrarily. It also supports training one part of a network with gradients and another with projections.▷ I am going to delete this and put it in the appendix

2 Related Work

Target propagation. Target propagation is a gradient-free learning paradigm where layer-wise targets are propagated backward through the network instead of gradients [11]. While these methods attempt to generate these targets by approximating the inverse of the forward map, our projection-based approach diverges by treating layer-wise updates as local feasibility problems. Instead of

¹While PJAX is built on JAX, it utilizes the framework solely as a high-performance tensor computation library; all neural network training logic has been completely redefined within PJAX itself

relying on approximate inverses. Ptorch computes targets by mathematically projecting the current state onto the layer’s constraint set to find the absolute smallest adjustment required to satisfy the network’s architectural constraints. Despite these different mechanisms for generating local targets, both approaches share a fundamental limitation: the attenuation of the learning signal with depth.

ADMM-based deep learning. The Alternating Direction Method of Multipliers (ADMM) offers a prominent alternative to the projection-based training of neural networks. Similar to projection-based methods, ADMM decouples the network into per-layer subproblems by introducing auxiliary variables and enforcing consistency constraints via a Lagrangian penalty. Wang et al. [18] advanced this line of work with the dlADMM algorithm, which updates parameters in a backward-then-forward sweep to propagate information more efficiently. Crucially, their approach reduces the subproblem complexity from $O(n^3)$ to $O(n^2)$ using iterative quadratic approximations and backtracking, while also establishing rigorous global convergence guarantees under mild conditions on the loss and activation functions. Ultimately, the layer-wise decomposition in dlADMM conceptually aligns with P Torch’s target propagation: both methods bypass end-to-end differentiation by decomposing the global objective into isolated, layer-wise optimization steps propagated through the computational graph.

Gradient-free optimization frameworks. A variety of other gradient-free optimization methods have been proposed in the literature, including the Method of Auxiliary Coordinates [3]. By unfolding the network into layers, this approach replaces a complex nested optimization problem with local optimization problems at the level of individual layers. Distinct paradigms such as neuroevolution have also attracted significant attention and even inspired dedicated frameworks [17]. Another notable alternative is the more recent Forward-Forward algorithm [10]. However, a common thread across these approaches is that none has yet achieved widespread adoption in mainstream deep learning. ▷ Unsure how to incorporate ProxProp [7]

3 Neural networks as a feasibility problem

3.1 The intuition behind the method

In contrast to gradient-based optimization, projection-based learning reframes training as a feasibility problem rather than a purely loss-minimization problem.

Imagine the neural network as a sequence of interlocking constraints. Every layer has a strict rule it must satisfy (e.g., “my output must exactly equal my weights multiplied by my input”). During a training step, we force the final output of the network to match the true target label. In doing so, we break the mathematical consistency of the layer immediately below it.

To restore consistency, we do not compute a gradient. Instead, we ask a simpler local question at each layer: “What is the smallest adjustment I can make to my weights, my incoming activations, and my output so that the constraint is perfectly satisfied again?”

By repeatedly applying these local projections sequentially from the output back to the input of the network, we propagate a learning signal without ever requiring gradients.

To get a better understanding of the algorithm a step by step walkthrough is p

3.1.1 local example

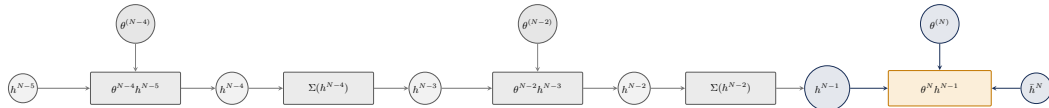


Figure 1: Update of the output linear layer: The projection acts as a mediator. It takes the rigid demand from the output target $\tilde{h}^N$ and distributes the required adjustments between the deepest hidden state h^{N-1} and the weights θ^N .

To see how this works mathematically, let’s look at how a single linear layer reacts to a target from above. Assume layer N is A linear layer with weights $\theta^{(N)}$.

We define this layer's constraint as all the possible combinations of inputs, weights, and outputs that have a possible matrix multiplication relation

$$A_N := \{(h^{N-1}, \theta^N, h^N) \mid h^N = \theta^N h^{N-1}\}.$$

When the network output is projected toward the true label, we receive a new target $\bar{h}$. The current weights and inputs no longer produce this target. To fix this, we project the current state onto the constraint set A_N and thus we get a new input h that satisfies this constraint but makes the previous layer not consistent anymore.

A full walkthrough is provided in appendix A

3.2 The mathematical description

Let $\{(x_1, y_1), \dots, (x_B, y_B)\}$ be a given dataset and suppose that we have a neural network with parameters $\theta_1, \dots, \theta_N$. For ease of presentation, we assume the network has a chain structure while an extension to more complex topologies is clear. In particular, the forward pass for a single sample (x_b, y_b) can be written as

$$\begin{aligned} h_b^{(0)} &= x_b \\ h_b^{(i+1)} &= f_{i+1}(h_b^{(i)}, \theta^{(i+1)}), \quad i = 0, \dots, N-1 \end{aligned}$$

for some functions $f_1, \dots, f_N$. The output of the network is given by h^N . For example, if θ^i is a matrix and Σ is an activation function, then $f_i(h_b^{(i-1)}, \theta^{(i)}) = \theta^i h_b^{(i-1)}$ followed by $f_{i+1}(h_b^{(i)}, \theta^{(i+1)}) = \Sigma(h_b^{(i)})$ is the standard building block of a multilayer perceptron. Note that $\theta^{(i+1)}$ is an "empty" variable in this case.

To model the network as a feasibility problem we can associate each datapoint (x_b, y_b) to the auxiliary variables $\Omega_i = \{h_b^{(0)}, \dots, h_b^{(N)}, \theta_b^{(1)}, \dots, \theta_b^{(N)}\}$. Then, there are three types of constraints that we consider:

1. **Architectural constraints:** each variable must be a feasible output of its corresponding neural network operation.
2. **Batch (data) constraints:** the inputs and outputs of the network are fixed to match the current training batch.
3. **Consensus constraints:** variables shared across multiple constraints must agree on a single consistent value.

Our feasibility problem is then to find some $\omega = (\Omega_1, \dots, \Omega_B)$ such that

$$\omega \in \left(\bigcap_{b=1}^B \bigcap_{i=1}^N A_{bi} \right) \cap (B_{\text{in}} \cap B_{\text{out}}) \cap \left(\bigcap_{i=1}^N C_i \right)$$

where A_{ij} denote the Architectural constraint sets, B_{in} and B_{out} denote the Batch constraint sets at the input and output respectively, and C_j denote certain Consensus constraint sets.

In particular, A_{bi} posits that $h_b^{(i)} = f_i(h_b^{(i-1)}, \theta_b^{(i)})$ for each $b = 1, \dots, B, i = 1, \dots, N$. Further, B_{in} requires $h_b^{(0)} = x_b$ while B_{out} requires $h_b^{(N)} = y_b$. Lastly, since we duplicated the neural network parameters $\theta^{(1)}, \dots, \theta^{(N)}$ for each datapoint, they must all equal each other, i.e., C_i ensures that $\theta_1^{(i)} = \dots = \theta_b^{(i)}$ for each $i = 1, \dots, N$. If the problem is feasible, then an equivalent minimization problem is

$$\omega^* \in \arg \min_{\omega \in \Omega} \underbrace{\frac{1}{2} \left\{ \sum_{b=1}^B \sum_{i=1}^N \text{dist}^2(A_{bi}, \omega) + \text{dist}^2(B_{\text{in}}, \omega) + \text{dist}^2(B_{\text{out}}, \omega) + \sum_{i=1}^N \text{dist}^2(C_i, \omega) \right\}}_{:=F(\omega)}$$

where $\text{dist}^2(C, \omega) = \min_{y \in C} \|\omega - y\|^2$.

To summarize, if we find an ω that resides in the intersection of these sets, then the parameters $\theta^{(1)}, \dots, \theta^{(N)}$ (which are equal over all datapoints due to the consensus constraint) allow the neural network to perfectly interpolate the data.

152

153 4 Cyclic projections

154 Our base algorithm to solve such a feasibility problem is cyclic projections. Starting from some initial
 155 parameters $h_1^{(0)}, \dots, h_1^{(N)}, \theta_1^{(1)}, \dots, \theta_1^{(N)}, \dots, h_B^{(0)}, \dots, h_B^{(N)}, \theta_B^{(1)}, \dots, \theta_B^{(N)}$ (for example, obtained
 156 from a single forward pass). Then the cyclic projections algorithm iteratively projects onto each
 157 of the sets. In the context of neural network training infrastructure, it is natural to overwrite the
 158 backward pass with these projections so that we repeat following algorithm until convergence.

159 This is possible because we assigned one projection to each functional node of the computational
 160 graph, allowing the forward pass to be identical to the gradient based case but in the backward pass
 we calculate the projection instead of the gradient each time we visit a node.

```

1:  $h_b^{(N)} \leftarrow y_b, b = 1, \dots, B$  ▷ Projection  $B_{\text{out}}$ 
2: for  $i = N - 1$  downto 0 do ▷ Traverses the computational graph backwards
3:   for  $b = 1$  upto  $B$  do
4:      $(\bar{h}_b^{(i)}, \bar{\theta}_b^{(i+1)}, \bar{h}_b^{(i+1)}) \in P_{A_{b,i+1}}(h_b^{(i)}, \theta_b^{(i+1)}, h_b^{(i+1)})$  ▷ Projection  $A_{b,i+1}$ 
5:      $(h_b^{(i)}, \theta_b^{(i+1)}, h_b^{(i+1)}) \leftarrow (\bar{h}_b^{(i)}, \bar{\theta}_b^{(i+1)}, \bar{h}_b^{(i+1)})$ 
6:   end for
7:    $\bar{\theta}^{(i+1)} \leftarrow \frac{1}{B} \sum_{b=1}^B \theta_b^{(i+1)}$ 
8:    $\theta_b^{(i+1)} \leftarrow \bar{\theta}^{(i+1)}$  ▷ Projection  $C_{i+1}$ 
9: end for
10:  $h_b^{(0)} \leftarrow x_b, b = 1, \dots, B$  ▷ Projection  $B_{\text{in}}$ 

```

161

162 4.0.1 connection to incremental gradient descent

163 To motivate the constructions in the following sections, we make explicit the connection between
 164 cyclic projections and the incremental gradient method, see e.g. Bertsekas [14]. Recall from Section ??
 165 that training reduces to the finite-sum minimization

$$\min_{\omega} F(\omega) := \frac{1}{2} \left\{ \sum_{b=1}^B \sum_{i=1}^N (\text{dist}^2(A_{bi}, \omega) + \text{dist}^2(B_{\text{in}}, \omega) + \text{dist}^2(B_{\text{out}}, \omega) + \sum_{i=1}^N \text{dist}^2(C_i, \omega)) \right\},$$

166 which has the form $F(\omega) = \sum_{j=1}^M f_j(\omega)$, with each component $f_j(\omega) = \text{dist}^2(C_j, \omega)$ a squared
 167 distance to a constraint set. Wherever the projection P_{C_j} is single-valued, f_j is differentiable with

$$\nabla f_j(\omega) = \omega - P_{C_j}(\omega). \quad (1)$$

168 A subtlety is that the constraint sets induced by the network are non-convex, so $P_{C_j}(\omega)$ need not be
 169 unique: at points ω that are equidistant from two or more elements of C_j , the projection is ambiguous
 170 and (1) is not well defined. This issue is benign for two reasons. First, the set of such points has
 171 Lebesgue measure zero, so (1) holds almost everywhere by Rademacher's theorem. Second, on this
 172 exceptional set the residual $\omega - P_{C_j}(\omega)$ remains a valid subgradient under any choice of $P_{C_j}(\omega)$, so
 173 the update direction is well-defined in either case. The situation is identical to that of ReLU at the
 174 origin: non-differentiable at a single point, yet trained without difficulty using a subgradient selection.
 175 A proof of (1) is given in Appendix ??.

176 With (1) in hand, the cyclic projection update of Algorithm ?? is exactly an incremental gradient step
 177 on F . The plain projection step

$$\theta^{(i+1)} \leftarrow P_{C_j}(\theta^{(i+1)}) \quad (2)$$

178 can be rewritten as

$$\theta^{(i+1)} \leftarrow (1 - \alpha) \theta^{(i+1)} + \alpha P_{C_j}(\theta^{(i+1)}), \quad \alpha = 1, \quad (3)$$

179 which is the relaxed projection of Section 4.3 with relaxation factor α . Rearranging (3) and substituting (1) gives

$$\theta^{(i+1)} \leftarrow \theta^{(i+1)} - \alpha(\theta^{(i+1)} - P_{C_j}(\theta^{(i+1)})) = \theta^{(i+1)} - \alpha \nabla f_j(\theta^{(i+1)}), \quad (4)$$

181 which is precisely the incremental gradient update [14, Eq. (2.78)]

$$\omega^{k+1} = \omega^k - \alpha^k \nabla f_{j_k}(\omega^k) \quad (5)$$

182 applied to the finite-sum objective (4.0.1), with step size α^k and index j_k chosen by the cyclic order in which Algorithm ?? traverses the computational graph. We are using the cyclic order index-selection rules listed in [14, §2.4.1] correspond directly to natural variants of our scheme.

185 This identification has two practical consequences. First, the relaxation factor α in Section 4.3 is not a free heuristic: it is the step size of an incremental gradient method on F , with $\alpha = 1$ recovering the plain projection and $\alpha = 2$ recovering.

186 To improve this base algorithm, improvements to the sensitive steps have been made ranging from an improved Matrix Multiplication Projection for Linear layers, the introduction of adam and muon as non linear relaxation heuristics and as introduced in PJAX the loss implemented through proximal projections[2]

192 4.1 Moreau-envelope view of the proximal loss step

193 ORIGINEEL IDEE VAN HIER + NOTATION IS AI GENERATED STILL[2] By substituting the rigid output assignment with a proximal mapping, we effectively perform an incremental gradient step on the Moreau envelope of the loss function. For a prediction h and target y , we define the Moreau envelope and the corresponding proximal operator directly on the loss $\mathcal{L}(\cdot, y)$:

$$e_{1/\lambda} \mathcal{L}(h, y) := \inf_u \left\{ \mathcal{L}(u, y) + \frac{\lambda}{2} \|u - h\|^2 \right\},$$

$$\text{prox}_{\mathcal{L}/\lambda}(h, y) := \arg \min_u \left\{ \mathcal{L}(u, y) + \frac{\lambda}{2} \|u - h\|^2 \right\}.$$

197 The proximal loss step updates the final network prediction h^N via $\bar{h}^N = \text{prox}_{\mathcal{L}/\lambda}(h^N, y)$. When $\mathcal{L}(\cdot, y)$ is ρ -weakly convex with $\lambda > \rho$, the envelope $e_{1/\lambda} \mathcal{L}(\cdot, y)$ is λ -Lipschitz-smooth, with its gradient given by

$$\nabla_h e_{1/\lambda} \mathcal{L}(h^N, y) = \lambda(h^N - \text{prox}_{\mathcal{L}/\lambda}(h^N, y)). \quad (6)$$

200 Thus, the update $h^N \leftarrow \bar{h}^N$ is exactly a gradient descent step of size $1/\lambda$ on the smoothed loss $e_{1/\lambda} \mathcal{L}$. The proximal-loss variant therefore optimizes a surrogate objective where the original loss is replaced by its Moreau envelope:

$$\tilde{F}_\lambda(\omega) = \Phi_{\text{net}}(\omega) + 2, e_{1/\lambda} \mathcal{L}(h^N(\omega), y), \quad (7)$$

203 where $\Phi_{\text{net}}(\omega)$ encapsulates all internal network constraints and distances. This recovers the exact (unsmoothed) target-assignment algorithm as $\lambda \rightarrow 0^+$.

205 Two consequences explain the empirical gains. First, $\nabla_h e_{1/\lambda} \mathcal{L}$ is λ -Lipschitz, so the output residual that initiates the backward sweep is bounded uniformly in h^N rather than scaling with $\|h^N - y\|$ as in the rigid case. Second, for convex $\mathcal{L}(\cdot, y)$ the smoothed and original problems share minimizers for every $\lambda > 0$ [rockafellar1998variational], so λ trades smoothness against surrogate tightness without moving the solution. Standard incremental gradient theory [14, §2.4] then applies to $\tilde{F}_\lambda$, and the weakly convex case is covered by Davis–Drusvyatskiy [davis2019stochastic].

```

1:  $h_b^{(N)} \leftarrow \text{prox}_{\lambda\ell}(h_b^{(N)}, y_b)$ ,  $b = 1, \dots, B$   $\triangleright$  Proximal step  $B_{\text{out}}$ ; replaces  $h_b^{(N)} \leftarrow y_b$ 
2: for  $i = N - 1$  downto 0 do  $\triangleright$  Traverses the computational graph
3:   ...
4: end for

```

4.2 Efficient Linear Layer

4.2.1 Combined consensus and bilinear constraint in ℓ_2

In standard projection-based training, the consensus projection for the linear layer is the primary memory bottleneck. A naive batched implementation requires building an $M \times K \times B$ tensor of rows (where M is the output dimension, B is the batch size, and K is the input dimension) before averaging. This $\mathcal{O}(MKB)$ memory footprint routinely causes out-of-memory errors on standard hardware.

To resolve this, we calculate the bilinear projection and the consensus linear projection simultaneously. By collapsing these steps, we eliminate the need to store a new set of weights for each individual sample within the batch loop, completely avoiding the $\mathcal{O}(MBK)$ memory blow-up.

Formally, this operation consists of two distinct projections: one onto the bilinear constraint set and one onto the consensus set.

For a sample b and output neuron j , the bilinear constraint $\theta^i[j, :]^\top h_b^i = h_b^{i+1}[j]$ is solved via Newton iteration to find a scalar t_{jb} (see Appendix ??). Substituting the shared weight θ^i for the per-sample copy, the projected row becomes linear in $\theta^i[j, :]$ and h_b^i :

$$\bar{\theta}_b^i[j, :] = \omega_{jb} \theta^i[j, :] + v_{jb} h_b^i{}^\top, \quad \text{where} \quad \omega_{jb} := \frac{\alpha}{\alpha - t_{jb}^2}, \quad v_{jb} := \frac{t_{jb}}{\alpha - t_{jb}^2}. \quad (8)$$

Second, the consensus projection forces all per-sample weights to agree by taking the batch average:

$$\bar{\theta}^i[j, :] = \frac{1}{B} \sum_{b=1}^B \bar{\theta}_b^i[j, :]. \quad (9)$$

We can directly substitute the bilinear projection into the consensus average and factor out the shared parameters:

$$\bar{\theta}^i[j, :] = \frac{1}{B} \left[\theta^i[j, :] \sum_{b=1}^B \omega_{jb} + \sum_{b=1}^B v_{jb} h_b^i{}^\top \right]. \quad (10)$$

By stacking the scalars ω_{jb} and v_{jb} into matrices $\omega, V \in \mathbb{R}^{M \times B}$, and stacking the hidden states into $H \in \mathbb{R}^{B \times K}$, we can assemble the update across all output rows simultaneously:

$$\bar{\Theta} = \frac{1}{B} (\Theta \odot (\omega \mathbf{1}_B) + VH). \quad (11)$$

The second term is evaluated as a single $M \times B$ by $B \times K$ matrix multiplication. Because the calculations are collapsed, no 3D intermediate tensor is ever formed. Peak memory usage drops efficiently from $\mathcal{O}(MBK)$ to $\mathcal{O}(MK + MB + BK)$.

We will apply this combined consensus and operation projection universally. If a mathematically efficient join cannot be found, the consensus will be handled implicitly within the layer itself, thereby allowing us to remove the for-loop over the samples.

4.2.2 ℓ_∞ implementation

The optimization geometry of the projection step can be modified to achieve different training properties. We replace the standard Euclidean projection with the ℓ_∞ one. To show the derivation of this we will be using a simplified notation: $\theta := \theta[j, :]$ $h^+ := h[j]^+$.

239 Given the current state $(h, \theta, h^+) \in \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R}$ and a scaling parameter $\gamma > 0$, the ℓ_∞ bilinear
 240 projection problem is

$$\min_{\epsilon} \epsilon \quad \text{s.t.} \quad \exists \bar{h}, \bar{\theta}, \bar{h}^+ : \begin{aligned} \|\bar{h} - h\|_\infty &\leq \epsilon \\ \|\bar{\theta} - \theta\|_\infty &\leq \epsilon \\ \gamma^2 |\bar{h}^+ - h^+| &\leq \epsilon \\ \bar{\theta}^\top \bar{h} &= \bar{h}^+ \end{aligned} \quad (12)$$

241 The key is finding the smallest feasible box radius $\epsilon^* \geq 0$ such that all constraints are simultaneously
 242 satisfiable.

243 Let $\Delta h = \bar{h} - h$ and $\Delta \theta = \bar{\theta} - \theta$. The constraints read

$$|\Delta h_k| \leq \epsilon, \quad |\Delta \theta_k| \leq \epsilon \quad (k = 1, \dots, n), \quad |\bar{h}^+ - h^+| \leq \frac{\epsilon}{\gamma^2}.$$

244 The total dot-product change decomposes coordinate-wise. Let $D = \theta^\top h$ and $\bar{D} = \bar{\theta}^\top \bar{h}$:

$$\begin{aligned} \bar{D} &= (\theta + \Delta \theta)^\top (h + \Delta h) \\ \Delta D_k &= (h_k + \Delta h_k)(\theta_k + \Delta \theta_k) - h_k \theta_k \\ &= h_k \Delta \theta_k + \theta_k \Delta h_k + \Delta h_k \Delta \theta_k. \end{aligned} \quad (13)$$

245 For any coordinate k , the extrema of ΔD_k over the box $[-\epsilon, \epsilon]^2$ are attained at the four corners
 246 $(\pm\epsilon, \pm\epsilon)$.

Δh_k	$\Delta \theta_k$	ΔD_k
$+\epsilon$	$+\epsilon$	$\epsilon(h_k + \theta_k) + \epsilon^2$
$-\epsilon$	$-\epsilon$	$-\epsilon(h_k + \theta_k) + \epsilon^2$
$+\epsilon$	$-\epsilon$	$\epsilon(\theta_k - h_k) - \epsilon^2$
$-\epsilon$	$+\epsilon$	$\epsilon(h_k - \theta_k) - \epsilon^2$

248 The maximum upward contribution of coordinate k to the dot product is

$$M_k(\epsilon) = \max\left(\epsilon|h_k + \theta_k| + \epsilon^2, \epsilon|h_k - \theta_k| - \epsilon^2\right). \quad (14)$$

249 In the case $D = \theta^\top h < h^+$, the smallest feasible radius ϵ^* is the unique positive root of

$$\begin{aligned} D + \sum_{k=1}^n M_k(\epsilon) &= h^+ - \frac{\epsilon}{\gamma^2}, \\ F(\epsilon) &= \sum_{k=1}^n M_k(\epsilon) + \frac{\epsilon}{\gamma^2} - (h^+ - D) = 0. \end{aligned} \quad (15)$$

250 F is continuous, strictly monotone increasing, and piecewise quadratic in ϵ .

251 Each coordinate contributes at most one kink to F , occurring at

$$\epsilon_k^{\text{kink}} = \max\left\{0, \frac{|h_k - \theta_k| - |h_k + \theta_k|}{2}\right\}.$$

252 Sorting these kink points requires $\mathcal{O}(n \log n)$ time, with root evaluation linear in n on each interval.

253 **$\mathcal{O}(n)$ Solve via Branch Masking** Define $a_k := |h_k + \theta_k|$ and $b_k := |h_k - \theta_k|$. Coordinate k
 254 contributes via whichever of two geometrically distinct regimes yields the larger reach:

- 255 • **Same-sign branch** ($a_k \epsilon + \epsilon^2$): the box pushes h_k and θ_k in the *same* direction, expanding
 256 their sum. This branch dominates when ϵ is large relative to the gap $b_k - a_k$.
- 257 • **Opp-sign branch** ($b_k \epsilon - \epsilon^2$): the box pushes h_k and θ_k in *opposite* directions, collapsing
 258 their difference. This branch dominates when ϵ is small.

259 The crossover between branches occurs at the unique kink $\epsilon_k^{\text{kink}} = \max\{0, (b_k - a_k)/2\}$. For ϵ
 260 above this threshold the same-sign branch wins; below it the opp-sign branch wins. This is encoded
 261 by the binary mask

$$m_k(\epsilon) := \mathbf{1}[2\epsilon \geq b_k - a_k],$$

262 so that the per-coordinate reach takes the *unified* branch-free form

$$M_k(\epsilon) = m_k(\epsilon)(a_k\epsilon + \epsilon^2) + (1 - m_k(\epsilon))(b_k\epsilon - \epsilon^2). \quad (16)$$

263 Since $m_k(\epsilon)$ is a step function that flips *at most once* as ϵ increases, both $F(\epsilon)$ and its derivative

$$F'(\epsilon) = \sum_{k=1}^n \left[m_k(\epsilon)(a_k + 2\epsilon) + (1 - m_k(\epsilon))(b_k - 2\epsilon) \right] + \frac{1}{\gamma^2} \quad (17)$$

264 are computable in a single $\mathcal{O}(n)$ pass by evaluating both branch expressions and selecting via m_k ,
 265 *without sorting or branching in control flow*. Because F is continuous, strictly increasing, and
 266 piecewise quadratic with a uniformly positive derivative (guaranteed by the $1/\gamma^2$ term), Newton's
 267 method is well-conditioned. The safeguarded iteration

$$\epsilon_{t+1} = \max\left(\ell_t, \min\left(r_t, \epsilon_t - \frac{F(\epsilon_t)}{F'(\epsilon_t)}\right)\right)$$

268 (with $[\ell_t, r_t]$ a bisection bracket maintained as a fallback) converges in $\mathcal{O}(1)$ iterations in practice: the
 269 piecewise-quadratic structure means each Newton step either lands in the correct quadratic segment
 270 immediately, or crosses at most one kink before the next iterate is already in the final segment. The
 271 full solve is therefore $\mathcal{O}(n)$.

272 4.3 Non-linear Relaxation

273 A practical advantage of the incremental-gradient interpretation in the previous subsection is that any
 274 first-order optimizer that uses gradients SGD with momentum, Adam, Muon can use the projection
 275 residual $\Delta\theta^{(i+1)}$ unchanged. This gives the empirical advantages of momentum and per-parameter
 276 scaling that drive modern gradient-based training, while leaving the underlying cyclic projection
 277 algorithm intact.

278 **Relaxed projection.** Plain cyclic projection overwrites the current iterate with the projected point.
 279 Relaxed variants standard in the convex optimization literature instead take a convex combination of
 280 the two.

Let $\bar{\theta}^{(i+1)} := P_{C_S}(\theta^{(i+1)})$ denote the projected iterate and define the relaxeprojection step

$$\theta^{(i+1)} \leftarrow \theta^{(i+1)} + \eta \Delta\theta^{(i+1)}. \quad (18)$$

Here $\eta = 1$ recovers the exact projection onto the feasible set, $\eta = 2$ produces a full reflection through that boundary, and $\eta \in (0, 1)$ yields under-relaxed steps. Over-relaxed (reflected) projections can converge faster but are less stable, while under-relaxed projections trade speed for stability—the same trade-off familiar from gradient-based step-size selection.

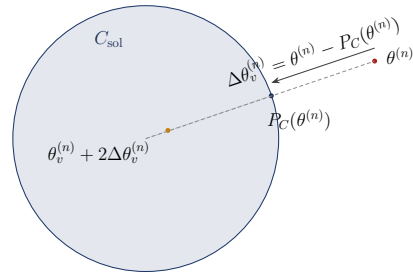


Figure 2: Geometry of pseudo-gradient updates. $\eta = 1$ reaches the exact projection; $\eta = 2$ produces a reflected step.

281 **Adaptive, per-parameter relaxation.** There is no reason to fix η to a single scalar. Since (4) shows
 282 that $\Delta\theta^{(i+1)}$ is a valid (sub)gradient of the finite-sum objective (4.0.1), we can let the relaxation factor
 283 vary across parameters and adapt over the course of training by feeding $\Delta\theta^{(i+1)}$ into a standard first-
 284 order optimizer in place of the gradient. The optimizer's internal state—momentum buffers in SGD
 285 with momentum, first- and second-moment estimates in Adam, the Newton–Schulz orthogonalization
 286 in Muon—then plays the role of an adaptive, per-parameter preconditioner replacing the scalar η .
 287 Because $\mathcal{P}$ Torch already routes projection residuals through PyTorch's autograd, no algorithmic

```

1: ...
2:  $(h_b^{(i)}, \theta_b^{(i+1)}, h_b^{(i+1)}) \leftarrow \text{Optimizer}(\bar{h}_b^{(i)} - h_b^{(i)}, \bar{\theta}_b^{(i+1)} - \theta_b^{(i+1)}, \bar{h}_b^{(i+1)} - h_b^{(i+1)})$   $\triangleright$ 
   Relaxed projection via pseudo-gradient
3: ...

```

modification is required: the residual is written into `.grad` and the optimizer is invoked exactly as it would be in a standard training loop.

In practice, this approach is most effective when the optimizer is applied only to the weight residuals $\bar{\theta}_b^{(i+1)} - \theta_b^{(i+1)}$, leaving the hidden-state residuals as plain projections. In the view of incremental gradient descent this means we are applying standard gradient-descent to the hidden states and accelerated gradient-descent to the weights.

4.4 Extension to Mini-Batches and the Full Algorithm

The base algorithm natively handles the full-batch case. To extend this approach to mini-batches, we introduce an inner loop that repeats the backward pass K times per batch. The hyperparameter K controls how strictly the network enforces the constraints of the current mini-batch before moving on to the next.

This introduces a natural optimization trade-off. A high value of K allows the network to tightly satisfy the current batch constraints, but it risks overfitting or stalling the training process. Conversely, a low value of K leads to noisier, more stochastic training dynamics, as the network rapidly alternates between the conflicting constraints of different mini-batches.

Algorithm 1 Mini-Batch $\mathcal{P}$ Torch

```

1:  $h_b^{(N)} \leftarrow \text{prox}_{\lambda\ell}(h_b^{(N)}, y_b)$ ,  $b = 1, \dots, B$ 
2: for  $k = 1$  to  $K$  do  $\triangleright$  Repeat the backward sweep  $K$  times
3:   for  $i = N - 1$  downto  $0$  do  $\triangleright$  Traverse the computational graph backward
4:      $(\bar{h}^{(i)}, \bar{\theta}^{(i+1)}, \bar{h}^{(i+1)}) \in P_{A_{i+1}}(h^{(i)}, \theta^{(i+1)}, h^{(i+1)})$ 
5:      $(h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \leftarrow \text{Optimizer}(\bar{h}^{(i)} - h^{(i)}, \bar{\theta}^{(i+1)} - \theta^{(i+1)}, \bar{h}^{(i+1)} - h^{(i+1)})$ 
6:   end for
7: end for
8:  $h_b^{(0)} \leftarrow x_b$  for  $b = 1, \dots, B$ 

```

Note that the explicit `for` loop over individual samples within a batch has been omitted. We assume that each projection operator is natively vectorized and processes the entire mini-batch in parallel, analogous to the behavior of standard deep learning frameworks.

A fundamental limitation of sequential target propagation in deep architectures is the progressive attenuation of the learning signal with depth. We formalize this observation below.

Assumption 4.1 (Local Non-Expansiveness). Let the architectural constraint set of layer $i + 1$ be

$$A_{i+1} := \{(h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \mid h^{(i+1)} = f_{i+1}(h^{(i)}, \theta^{(i+1)})\}.$$

Although A_{i+1} is generally non-convex due to nonlinearities and bilinear weight–activation interactions, we assume that for sufficiently close sets, the projection $P_{A_{i+1}}$ behaves non-expansively within a local neighborhood of the current iterates. This assumption is empirically verified in experiments.

Definition 4.2 (Layer-wise Target Signal). At training step n , the *target signal magnitude* at hidden layer i is

$$\delta^{(i),n} := \|\bar{h}^{(i),n} - h^{(i),n}\|_2.$$

5 Theoretical Analysis: Vanishing Target Signals in Deep Networks

Theorem 5.1 (Vanishing Target Signal). *Under Assumption 4.1, the target signal magnitude $\delta^{(i),n}$ is monotonically non-increasing as the propagation moves backward from the output layer N . Formally,*

$$\delta^{(i),n} \leq \delta^{(i+1),n} \quad \text{for all } 0 \leq i \leq N-1.$$

Consequently, for any network in which learning occurs (i.e., $\|\Delta\theta^{(i+1)}\| > 0$), the contraction is strict.

Proof. We proceed by induction moving backward from the output layer N .

Base case. At the output layer N , the proximal update for an arbitrary loss function $\mathcal{L}$ gives

$$\bar{h}^{(N),n} \in \arg \min_h \left(\mathcal{L}(h, y_{\text{target}}^n) + \frac{\lambda}{2} \|h - h^{(N),n}\|_2^2 \right).$$

Assuming $\mathcal{L}$ is proper and lower semi-continuous, this minimum is attained and bounded, so $\delta^{(N),n} = \|\bar{h}^{(N),n} - h^{(N),n}\|_2 < \infty$.

Inductive step. Suppose $\delta^{(i+1),n}$ is finite. Consider the two points in the joint space $(h^{(i)}, \theta^{(i+1)}, h^{(i+1)})$:

$$A; =; (h^{(i),n}, ; \theta^{(i+1),n}, ; \bar{h}^{(i+1),n}), B \quad ; =; (h^{(i),n}, ; \theta^{(i+1),n}, ; h^{(i+1),n}).$$

Since the forward pass satisfies the constraint exactly, $B \in A_{i+1}$ and hence $P_{A_{i+1}}(B) = B$. By Assumption 4.1:

$$\|P_{A_{i+1}}(A) - P_{A_{i+1}}(B)\|_2^2 \leq \|A - B\|_2^2.$$

Expanding both sides and using the fact that A and B agree in their first two components:

$$\|\bar{h}^{(i),n} - h^{(i),n}\|_2^2 + \|\Delta\theta^{(i+1)}\|_2^2 + \|h^{(i+1),n+1} - h^{(i+1),n}\|_2^2 \leq \|\bar{h}^{(i+1),n} - h^{(i+1),n}\|_2^2,$$

which gives $(\delta^{(i),n})^2 \leq (\delta^{(i+1),n})^2 - (\|\Delta\theta^{(i+1)}\|_2^2 + \|\Delta h^{(i+1)}\|_2^2) \leq (\delta^{(i+1),n})^2$, and therefore $\delta^{(i),n} \leq \delta^{(i+1),n}$.

By induction this generalizes to $\delta^{(i),n} \leq \delta^{(i+1),n}$ for all i . When $\|\Delta\theta^{(i+1)}\|_2^2 > 0$ at every layer (i.e., the network is actively learning), each term is strictly smaller than the previous one. $\square$

Remark 5.2 (Depth Barrier). In a network that is actively learning, cyclic projections result in a diminishing learning signal to early layers. For networks of sufficient depth, the target difference at the earliest layers reaches machine precision, rendering parameter updates in those layers negligible.

6 Experiments

Unless otherwise noted, all benchmark scripts share the same experimental configuration: random seed 43, at most 50,000 optimization steps, 3 runs per setting, evaluation every 200 steps, patience 10, batch size 32, and $\mathcal{P}$ Torch restricted to the ℓ_∞ norm. PJAX uses the DouglasRachford optimizer with 50 projection steps per batch². $\mathcal{P}$ Torch uses ProjectionMuon with learning rate 0.001 and CrossEntropyProjection, and the gradient baseline uses Adam with learning rate 0.001.

6.1 Shallow MLP: MNIST and CIFAR-10

Setup. A single-hidden-layer MLP with 512 units is trained on MNIST and CIFAR-10. All three frameworks use identical hyperparameters.

²These are the settings used in the original paper

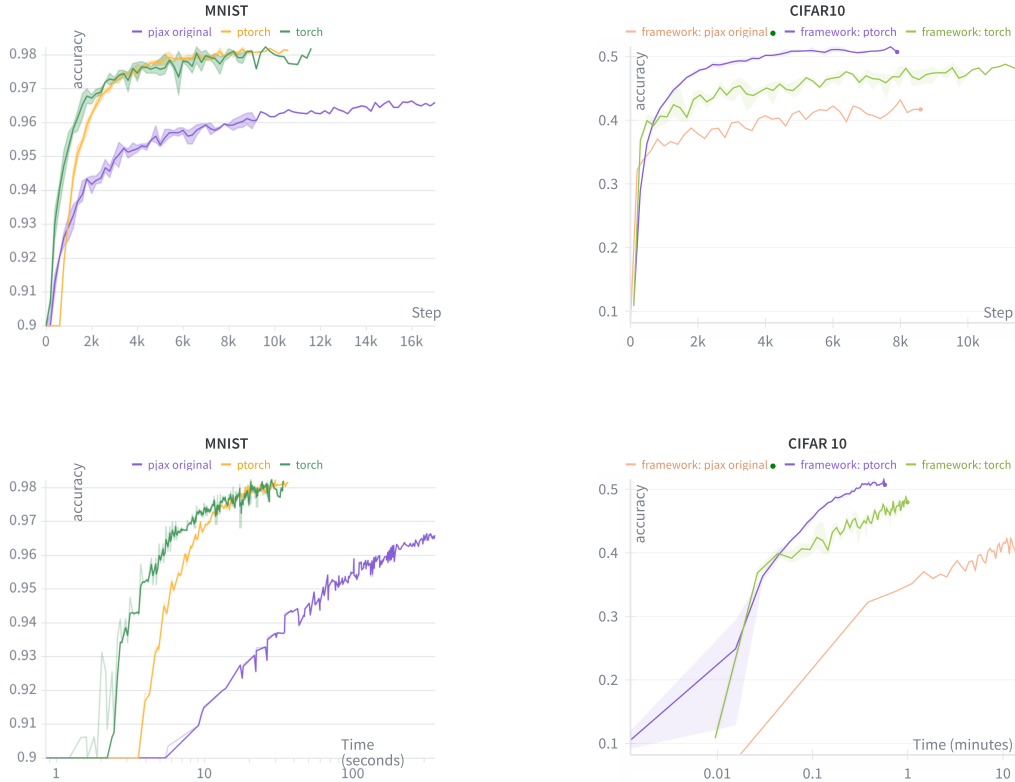


Figure 3: Single-hidden-layer MLP (512 units) on MNIST (left) and CIFAR-10 (right). Top row: accuracy vs. optimization step. Bottom row: accuracy vs. wall-clock time. $\mathcal{P}$ Torch reaches comparable final accuracy to torch while outpacing pjax per wall-clock second.

344 **Results.** These results demonstrate that $\mathcal{P}$ Torch is highly competitive with torch while significantly
 345 outperforming pjax.

346 However, it is important to note that these benchmarks were conducted with a batch size of 32. At
 347 smaller batch sizes, $\mathcal{P}$ Torch extends its lead further. Conversely, as the batch size scales upward,
 348 torch begins to see better performance gains, whereas pjax experiences a significant decline in speed.

349 6.2 Vanishing Target Signal and Depth

350 **Setup.** Following Definition 4.2, we train $\mathcal{P}$ Torch networks of depth $L \in \{2, 4, 8, 16\}$ (width 128,
 351 no activations, ℓ_∞ norm) to approximate the identity function $f(x) = x$ on $x \sim \mathcal{N}(0, I)$, recording
 352 δ_k^n at the first training step. A 4-layer MLP (4×512) is additionally evaluated on MNIST and
 353 CIFAR-10 under the same protocol as Section 6.1.

354 **Results.** Figure 4 (bottom) illustrates the training dynamics for 4-layer MLP architectures on the
 355 MNIST and CIFAR-10 datasets. Consistent with the limitations foreshadowed by our preliminary
 356 experiments, these results demonstrate that $\mathcal{P}$ Torch scales poorly as depth increases. In stark contrast
 357 to the shallower networks evaluated in Section 6.1 where $\mathcal{P}$ Torch maintained competitive performance
 358 the 4-layer models exhibit a severe degradation in learning.

359 Further empirical analysis (left) examines the residual difference between the target after projection
 360 and the current state. This delta diminishes exponentially, effectively leaving layers closest to the
 361 input without meaningful updates. Compounding this issue, the network struggles to learn the identity
 362 function in deeper configurations.

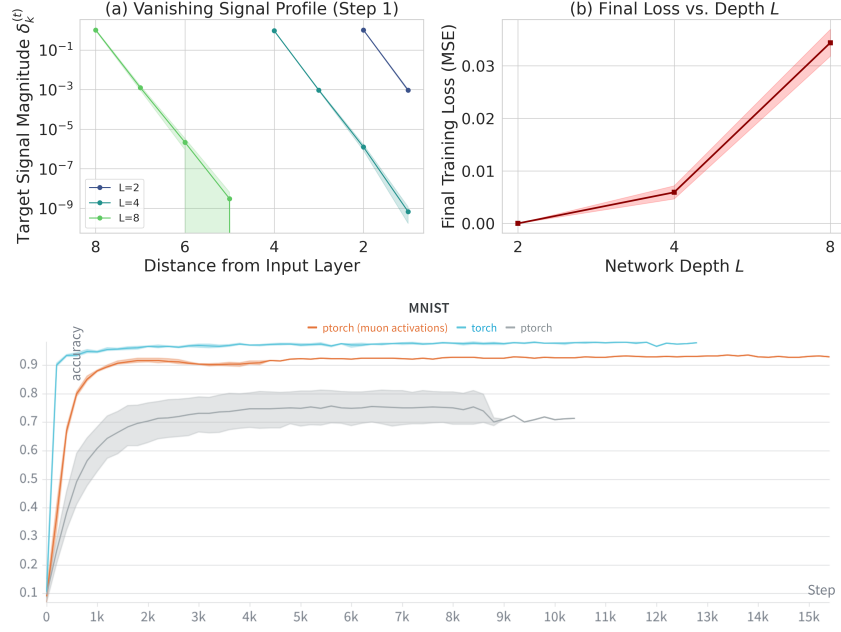


Figure 4: **Top:** per-layer target signal magnitude δ_k^n for varying depths (left) and final training loss vs. depth (right), confirming Theorem 5.1. **Bottom:** accuracy vs. wall-clock time for a 4-layer MLP on MNIST (left) and CIFAR-10 (right). The performance gap relative to torch widens with depth, consistent with Remark 5.2.

6.3 Modeling the loss as a proximal operator

Modeling the loss as a hard constraint vs proximal operator has a significant influence on the final performance of the algorithm. In this benchmark we look at a couple variants to see which performs best.

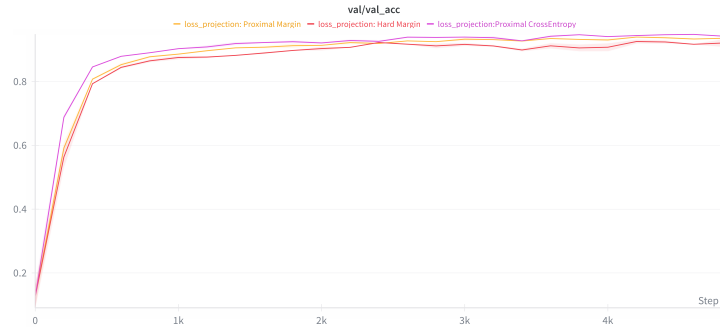
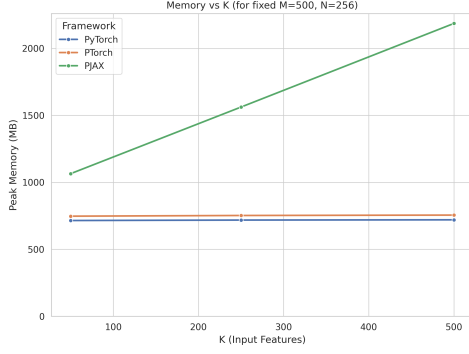


Figure 5: Applying A proximal operator instead of a hard projection improves performance

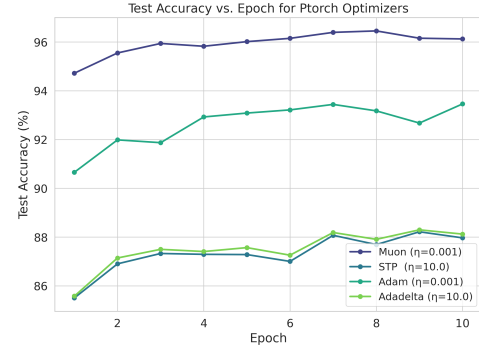
6.4 Matrix Multiplication & Optimizer Influence

Results. Using an adaptive optimizer yields substantial gains compared to standard sequential target propagation. Note that Adam’s instability under pseudo-gradients is not visible in these per-epoch accuracy figures; in contrast, Muon demonstrates superior stability despite a marginally slower convergence rate during the initial steps.

As predicted by the theory, our matrix-multiplication projection scales as $\mathcal{O}(MN)$, whereas pjax scales as $\mathcal{O}(MNK)$, making PTorch substantially more practical on real workloads where memory is a limiting factor.



(a) Peak memory vs. input features for a single linear layer. `pjax` scales as $\mathcal{O}(MNK)$; `PTorch` reduces this to $\mathcal{O}(MN)$, matching `torch`.



(b) Training loss vs. epoch for four optimizers applied to `PTorch` pseudo-gradients. Muon and momentum-based methods converge faster and to lower loss, consistent with the motivation in Section ??.

Figure 6: **Left:** Peak memory scaling for a single linear layer with 500 out features and 256 batch size as the input dimension grows. **Right:** Optimizer convergence comparison on the MNIST MLP (width 256), using the best learning rate per optimizer from a hyperparameter sweep.

375 6.4.1 optimal application location of the non linear relaxation

376 To answer the question whether we must apply the FO to all hidden states or only the weights or not at
 377 all, an xor classification network was trained with this applied on various locations

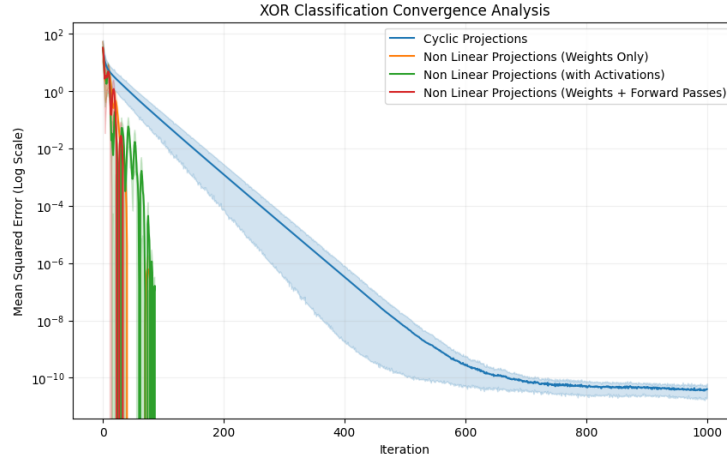


Figure 7: Applying non linear relaxation through adam and muon results better performance, especially if only applied to the weights.

378 6.4.2 Orthogonalization of activations

379 A particularly interesting approach is applying muon newton-schulz orthogonalization without
 380 rescaling on the hidden states and any other optimizer for the weights. This significantly improves
 381 the training performance of the network on deeper networks as the difference in targets becomes
 382 constant because of the orthogonalization step

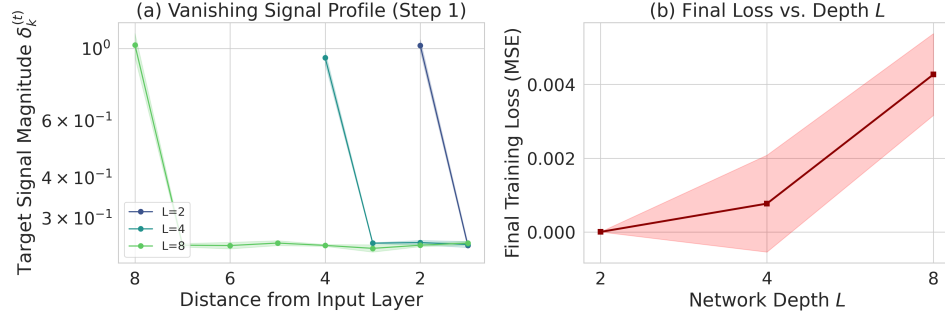


Figure 8: Applying the orthogonalization of muon onto the hidden states improves performance on deep networks but slightly reduces performance on shallow network.

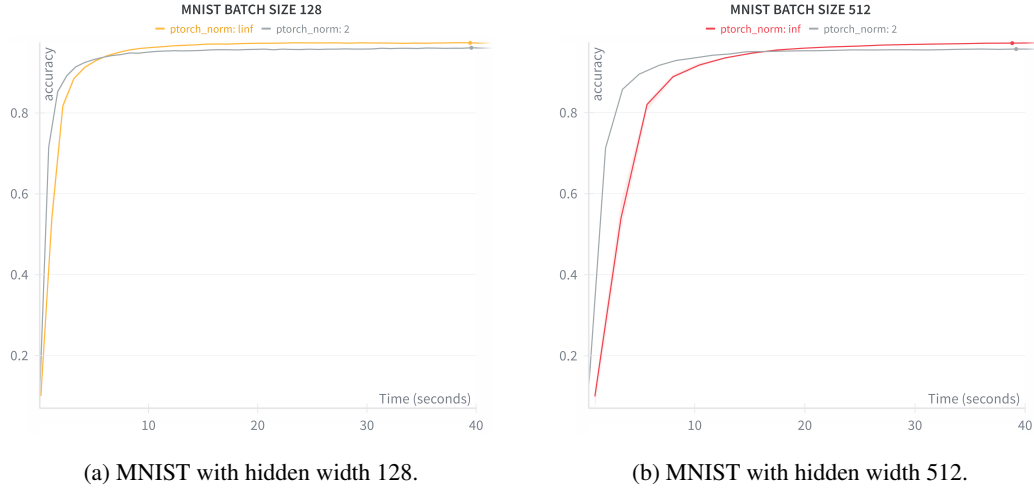


Figure 9: Comparison of matrix-multiplication projection norms on MNIST. The ℓ_∞ projection converges more slowly per optimization step but reaches higher final accuracy. As the batch size increases, the efficient ℓ_2 implementation becomes substantially faster than the ℓ_∞ projection; however, over longer training runs, the ℓ_∞ projection still achieves better final performance.

6.4.3 Influence of Projection Norm

6.5 Dykstras algorithm

As our current algorithms solve the feasibility of the issue and do not find the closest point in the set, a benchmark with dykstra's algorithm was performed to see if its beneficial to get the closest feasible point instead of just a feasible point. Note that this was done in pjax instead of PTorch as pjax splits the constraints into 2 sets while pjax uses N sets making pjax the more logical choice for this benchmark

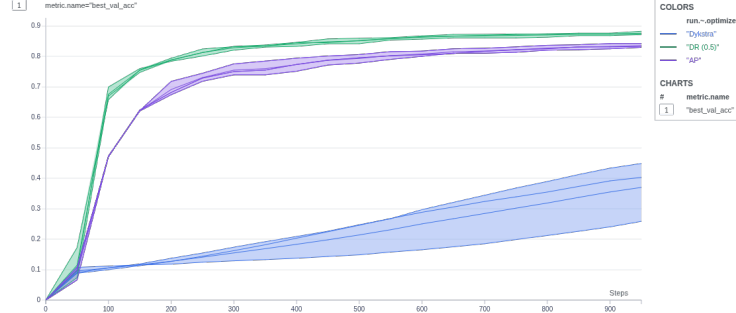


Figure 10: Dykstra’s algorithm while it has better theoretic guaranties for convex sets, in the case of non convex sets for neural network optimization it performs terribly

6.6 More architectures

Benchmarks using different common and uncommon architectures to better understand $\mathcal{P}$ Torch’s performance.

6.6.1 Convolutional neural networks

Given the previously established degradation of projection-based methods in deeper architectures, evaluating highly parameterized models such as standard, deep ResNets remains challenging. However, to demonstrate the practical viability and superior memory efficiency of $\mathcal{P}$ Torch compared to pjax , we evaluate a shallower convolutional network. Specifically, we employ a ResNet-6 architecture. This provides a baseline that is shallow enough to ensure convergence, yet demanding enough to highlight the memory limitations of prior projection implementations, which is unable to run this configuration on similar hardware. $\triangleright$ TEST THIS³

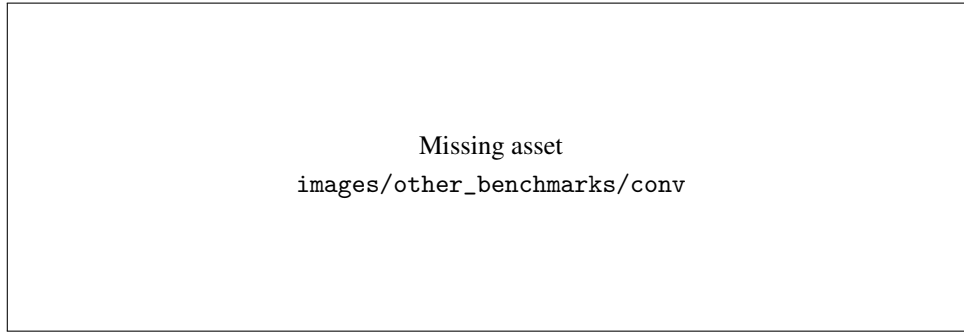


Figure 11: Training dynamics on the ResNet-6 architecture. While $\mathcal{P}$ Torch successfully optimizes convolutional networks that is memory-prohibitive for pjax , it exhibits much slower convergence and lower final accuracy compared to (torch) . This performance gap is consistent with the limitations associated with network depth

6.6.2 Quantized ReLu and the gapped step activation function

To demonstrate our framework’s capacity to directly optimize through non-differentiable activation functions without relying on heuristic approximations, we first evaluate a Boolean XOR classifier utilizing a gapped step function. To extend this evaluation to a more practical setting, we subsequently train an image classifier on the MNIST dataset using a quantized ReLU activation. Specifically, this quantized function outputs zero for inputs $x < 0$ and increases in discrete increments of for $x > 0$.

³This benchmark was highly optimized and shows the top performance of $\mathcal{P}$ Torch on deep networks, in general worse performance must be expected



(a) Boolean XOR with a gapped step function.

(b) MNIST with Quantized ReLU.

Figure 12: Optimization of networks with non-differentiable activation functions using $\mathcal{P}$ Torch. (a) Training convergence on the Boolean XOR problem utilizing a discontinuous gapped step function. (b) Performance on the MNIST dataset using a Quantized ReLU activation, which outputs zero for $x \leq 0$ and increases in discrete steps of size δ for positive inputs. Both experiments demonstrate the framework’s capability to directly and effectively optimize through hard discontinuities without relying on gradient approximations such as the Straight-Through Estimator (STE).

407 6.6.3 residual neural networks

408 Residual networks are sadly not yet very elegant in $\mathcal{P}$ Torch and require $\mathcal{P}$ Torch + a custom torch
 409 backend. The reason for this is that gradients are automatically summed in torch, yet for targets we
 410 require averaging. A modified version of torch is provided but requires a compile that takes a couple
 411 of hours. As reported by pjax [2] performance improves significantly when residual networks are used.

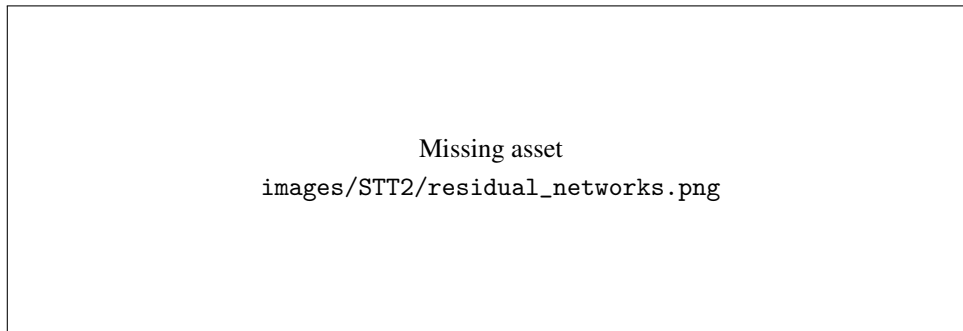


Figure 13: Performance when applying residual networks, you can train deep networks with this:)

412

413 6.6.4 constrained weight



Figure 14: RMS, layernorm, projections are very hard to define and usually make performance

414 A A Walk Through Cyclic Projections

415 This section unrolls Algorithm ?? for one batch on a feedforward network, making explicit how the
 416 abstract projection $P_{A_{b,i+1}}$ instantiates for the two canonical building blocks of an MLP (a linear
 417 sublayer and an elementwise activation), and how the proximal output step of Section 4.1 and the
 418 first-order relaxation of Section ?? slot into the per-layer updates.

419 A.1 Forward and Backward Semantics

420 Consider a feedforward network with N layers as in Section ??, with computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.
 421 Each node $v \in \mathcal{V}$ corresponds to a layer function f_i acting on the previous hidden state and (possibly
 422 empty) parameters,

$$h^{(i)} = f_i(h^{(i-1)}, \theta^{(i)}), \quad i = 1, \dots, N,$$

423 with $h^{(0)} = x$ and network output $h^{(N)}$. To exploit the native autograd engine, $\mathcal{P}$ Torch redefines
 424 the semantics of the forward and backward passes for each node.

425 **Definition A.1** (Training-Step Iterates). At training step n , the *forward pass* produces the current
 426 hidden-state and parameter iterates $\{h^{(i),n}\}_{i=0}^N$ and $\{\theta^{(i),n}\}_{i=1}^N$. The *backward pass* produces
 427 intermediate targets $\{\bar{h}^{(i),n}, \bar{\theta}^{(i),n}\}$ and the next iterates $\{h^{(i),n+1}, \theta^{(i),n+1}\}$.

428 For ease of presentation we suppress the batch index b throughout this section; the extension is the
 429 per-sample procedure of Algorithm ??, with the consensus step C_i averaging $\bar{\theta}_b^{(i),n}$ across the batch
 430 as in lines 7–8 of that algorithm.

431 A.2 Loss Projection and Target Initialization

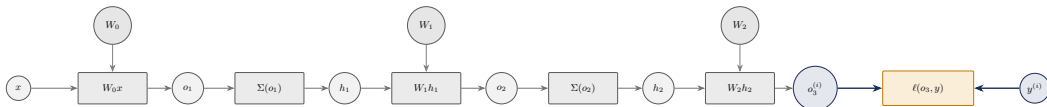


Figure 15: Initialization of the backward sweep: the network output $h^{(N),n}$ is coupled to the loss constraint via the proximal operator of Section 4.1, replacing the rigid assignment $h^{(N)} \leftarrow y$ of line 1 of Algorithm ??.

432 Let y^n denote the data target for batch n . The backward pass is initialized by the proximal step on
 433 the output layer.

434 **Definition A.2** (Output Target). The target for the network output $\bar{h}^{(N),n}$ is the proximal step

$$\bar{h}^{(N),n} = \text{prox}_{\mathcal{L}/\lambda}(h^{(N),n}, y^n) \in \arg \min_u \left(\mathcal{L}(u, y^n) + \frac{\lambda}{2} \|u - h^{(N),n}\|_2^2 \right). \quad (19)$$

435 This $\bar{h}^{(N),n}$ replaces the hard projection onto the data set $B_{\text{out}} = \{h^{(N)} = y\}$ used in the base cyclic
 436 algorithm (line 1 of Algorithm ??); it is then propagated to the preceding layer as the target $\bar{h}^{(N),n}$
 437 entering the projection P_{A_N} .

438 A proximal operator is used here rather than a hard projection onto B_{out} to accommodate loss
 439 functions for which the solution set is difficult to characterize explicitly. For instance, it is not
 440 immediately clear how to define the feasible set associated with binary cross-entropy, let alone how to
 441 project onto it directly. By contrast, hard-margin constraint sets have a natural geometric interpretation
 442 and admit straightforward projections. The proximal formulation thus provides a unified treatment
 443 that remains tractable even when direct projection is ill-defined; as shown in Section 4.1, it also
 444 corresponds to a gradient step on the Moreau envelope $e_{1/\lambda}\mathcal{L}$.

445 A.3 Layer-wise Propagation

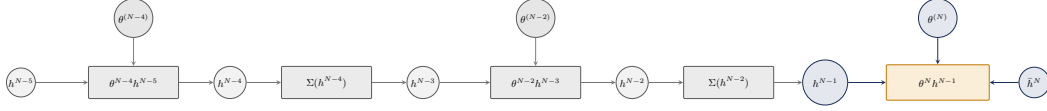


Figure 16: Update of the output linear layer: projection onto the architectural constraint A_N couples the output target $\bar{h}^{(N),n}$ to the deepest hidden state $h^{(N-1),n}$ and the weights $\theta^{(N),n}$.

446 Assume layer N is linear with weights $\theta^{(N)}$, so $f_N(h^{(N-1)}, \theta^{(N)}) = \theta^{(N)} h^{(N-1)}$. The correspond-
 447 ing architectural constraint set is

$$A_N := \{(h^{(N-1)}, \theta^{(N)}, h^{(N)}) \mid h^{(N)} = \theta^{(N)} h^{(N-1)}\}.$$

448 **Proposition A.3** (Linear-Layer Update). *Given the current iterate $(h^{(N-1),n}, \theta^{(N),n}, h^{(N),n})$ and*
 449 *the output target $\bar{h}^{(N),n}$ from Definition A.6, the projection*

$$(\bar{h}^{(N-1),n}, \bar{\theta}^{(N),n}, \bar{h}^{(N),n}) \in P_{A_N}(h^{(N-1),n}, \theta^{(N),n}, \bar{h}^{(N),n}) \quad (20)$$

450 *simultaneously yields the target hidden state $\bar{h}^{(N-1),n}$ and the projected weights $\bar{\theta}^{(N),n}$. Following*
 451 *Section ??, the next-step parameter iterate is obtained by feeding the projection residual into the*
 452 *chosen first-order optimizer:*

$$\theta^{(N),n+1} = \text{Optimizer}(\theta^{(N),n}, \eta, \theta^{(N),n} - \bar{\theta}^{(N),n}), \quad (21)$$

453 *with relaxation/step-size η . For plain STP this reduces to the exact projection $\theta^{(N),n+1} = \bar{\theta}^{(N),n}$.*
 454 *The target $\bar{h}^{(N-1),n}$ then becomes the output target entering the projection $P_{A_{N-1}}$ of the preceding*
 455 *(activation) layer.*

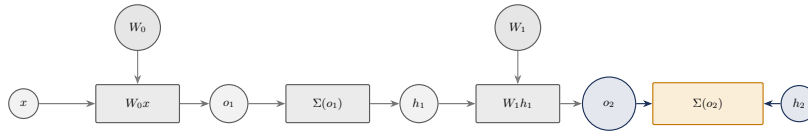


Figure 17: Projection through the activation constraint A_{N-1} , which carries no parameters.

456 Now consider the preceding activation layer $f_{N-1} = \sigma$, possibly non-differentiable. As noted in
 457 Section ??, $\theta^{(N-1)}$ is empty for activation layers, so the architectural constraint reduces to a relation
 458 between two hidden states:

$$A_{N-1} := \{(h^{(N-2)}, h^{(N-1)}) \mid h^{(N-1)} = \sigma(h^{(N-2)})\}.$$

459 **Proposition A.4** (Activation-Layer Update). *Given the target $\bar{h}^{(N-1),n}$ inherited from Proposi-*
 460 *tion A.7, the projection*

$$(\bar{h}^{(N-2),n}, \bar{h}^{(N-1),n}) = P_{A_{N-1}}(h^{(N-2),n}, \bar{h}^{(N-1),n}) \quad (22)$$

461 *yields the target $\bar{h}^{(N-2),n}$ for the preceding layer.*

462 This sequential procedure repeats for $i = N - 2, N - 3, \dots, 1$, alternating Propositions A.7 and A.8
 463 as the layer types dictate, until the input data constraint $B_{\text{in}} : h^{(0)} = x$ closes the sweep (line 10 of
 464 Algorithm ??).

A.4 Forward and Backward Semantics

Consider a neural network structured as a computational chain $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. To exploit the native autograd engine, $\mathcal{P}$ Torch redefines the semantics of the forward and backward passes for each node $v \in \mathcal{V}$.

Definition A.5 (Training Step Variables). During training step n , for each node v we denote its input by h_v and its output by q_v , so that the output of one node feeds the input of the next: $h_v = q_{v-1}$. The *forward pass* computes the current iterates (h^n, q^n) . The *backward pass* computes intermediate targets $(\bar{h}^n, \bar{q}^n)$ and the subsequent iterates (h^{n+1}, q^{n+1}) .

A.5 Loss Projection and Target Initialization

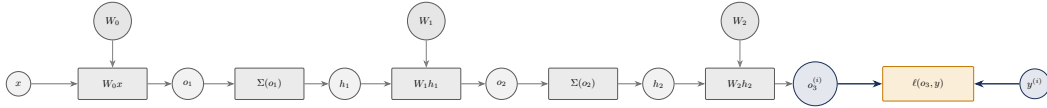


Figure 18: First projection of the backward pass: coupling the network output to the loss constraint.

Let y^n denote the data target for batch n . The backward pass is initialized by projecting the output onto the loss-minimizing constraint.

Definition A.6 (Output Target). The loss layer input target $\bar{h}_{L+1}^n$ is defined via the proximal operation

$$\bar{h}_{L+1}^n \in \arg \min_h \left(\mathcal{L}(h, y_{\text{target}}^n) + \frac{\lambda}{2} \|h - h_{L+1}^n\|_2^2 \right). \quad (23)$$

The target for the output layer is set to $q_L = \bar{h}_{L+1}^n$, which is then passed directly as the target to the preceding layer.

A proximal operator is used here rather than a projection to accommodate loss functions for which the solution set is difficult to characterize explicitly. For instance, it is not immediately clear how to define the feasible set associated with binary cross-entropy, let alone how to project onto it directly. By contrast, hard-margin constraint sets have a natural geometric interpretation and admit straightforward projections. The proximal formulation thus provides a unified treatment that remains tractable even when direct projection is ill-defined.

A.6 Layer-wise Propagation

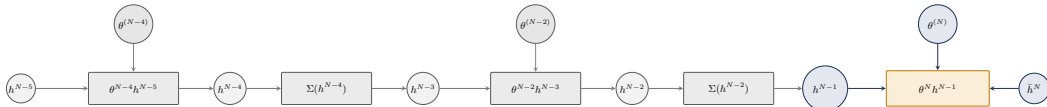


Figure 19: Update of the output weights: coupling the output loss with the deepest hidden layer.

The target output $\bar{q}_L^n = \bar{h}_{L+1}^n$ propagates to the preceding layer. The target input $\bar{h}_L^n$ and updated weights W_L^{n+1} are obtained by projecting onto the feasibility set:

Proposition A.7 (Linear Layer Update). Define the bilinear feasibility set $C_{\text{lin}} := \{(h, W, q) \mid q = Wh\}$. The projection

$$(\bar{h}_L^n, \bar{W}_L^n, \bar{q}_L^{n+1}) \in P_{C_{\text{lin}}}(h_L^n, W_L^n, \bar{q}_L^n) \quad (24)$$

simultaneously yields the subsequent output iterate q_L^{n+1} and the target input $\bar{h}_L^n$ for the layer. Since the input of layer L is the output of layer $L-1$, we have $\bar{q}_{L-1}^n = \bar{h}_L^n$. The updated weights are obtained by applying the optimizer to the pseudo-gradient direction, $W_L^{n+1} = \text{Opt}(W_L^n, \eta, W_L^n - \bar{W}_L^n)$ with step size η . For regular STP, this is the exact projection update $W_L^{n+1} = \bar{W}_L^n$.

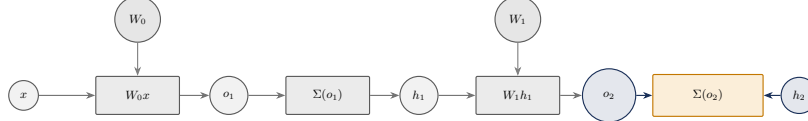


Figure 20: Projection through the nonlinear activation constraint.

Proposition A.8 (Activation Layer Update). *Let σ be a (possibly non-differentiable) activation function and define the activation feasibility set $C_\sigma := \{(h, q) \mid q = \sigma(h)\}$. Given the target $\bar{q}_{L-1}^n$ from Proposition A.7, the projection*

$$(\bar{h}_{L-1}^n, q_{L-1}^{n+1}) = P_{C_\sigma}(\bar{h}_{L-1}^n, \bar{q}_{L-1}^n) \quad (25)$$

yields the subsequent output iterate q_{L-1}^{n+1} and the target input $\bar{h}_{L-1}^n$, which becomes the target output $\bar{q}_{L-2}^n$ for the preceding layer.

This sequential projection procedure repeats, propagating targets backward through the computational graph $\mathcal{G}$ until the root is reached.

This section summarizes the projection operators used in $\mathcal{P}\text{TORCH}$. Most constructions are adapted from [2, 5]. Table ?? provides a structured overview.

B Tuning projection based training

To make projection-based neural network training algorithms more accessible for future researchers, this section is dedicated to discussing implementation and tuning details to ensure successful experiments.

B.0.1 Tuning order

Selecting the right optimizer is the most critical decision in this framework. In nearly all scenarios, the Muon projection optimizer yields the best results and is the recommended default. If you require faster initial convergence, SGD is a better alternative. Adams performance has a lot of potential but is very unstable and thus if you use it prepare to fine tune the learning rate a bit before its stable.

When utilizing SGD or Adadelta, higher learning rates even values such as 10, tend to perform better. A reliable tuning strategy is to start at 1 and adjust upward or downward based on training stability.

Regarding batch sizes: when using the ℓ_2 projection, larger batch sizes such as 256 or 512 generally improve results. Scaling beyond this point has not shown further benefits and may degrade performance. Conversely, if the ℓ_{inf} linear layer is used, a smaller batch size of 32 has proven more effective in testing.

When designing architectures for this framework, prioritize shallow networks, as they outperform deeper architectures and maintain better stability. Widths of up to 1024 have been tested and exhibit very rapid convergence. Adding Batch Normalization or similar normalization layers significantly degrades performance and should currently be avoided.

Furthermore, take special care to ensure all layers influencing the gradient are implemented via $\mathcal{P}\text{Torch}$ modules. Using PyTorch layers, for example their linear layer will lead to autograd getting confused as either it tries to process targets as gradients or vice versa. To implement custom architectures like residual connections, refer to the implementations in the `overrides.py` or residual networks benchmark file.

B.0.2 Projection parameters

The behavior of the projection can be fine-tuned by adjusting the α and g parameters, which determine whether the weights or activations absorb the bulk of the updates. Setting g to `inf` results in the output values not being changed, setting α to it will result in the weights not being changed. While the framework is robust and generally insensitive to these changes, precise adjustment can yield marginal gains when maximizing accuracy is sought. using an alpha and g in the range of [0.01-500] are logical values. The result of a hyperparameter tune can be seen below

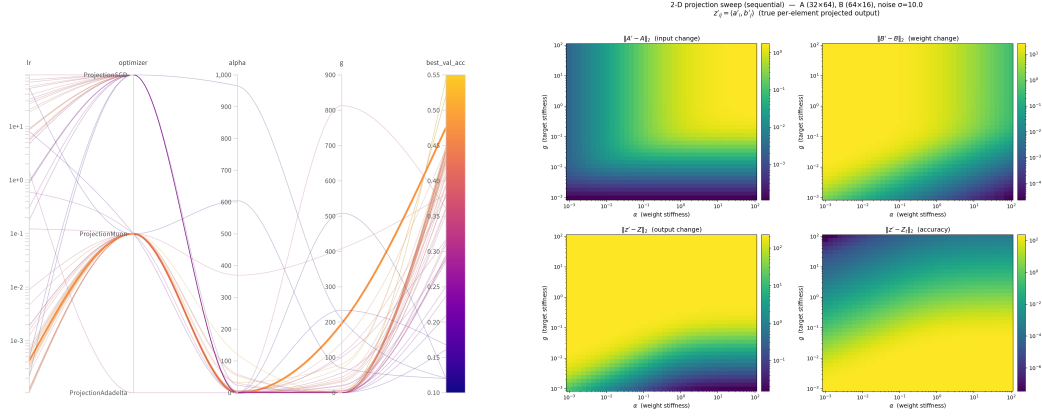


Figure 21: Influence of gamma and α parameters; training stability remains high across various settings

534 B.0.3 proper initialization

535 For initialization the same methods as gradient based approaches are used, this does is an active
 536 shortcoming of the framework as testing has indicated that the intialization of netork is extermly
 important for the convergence and final state of the network.

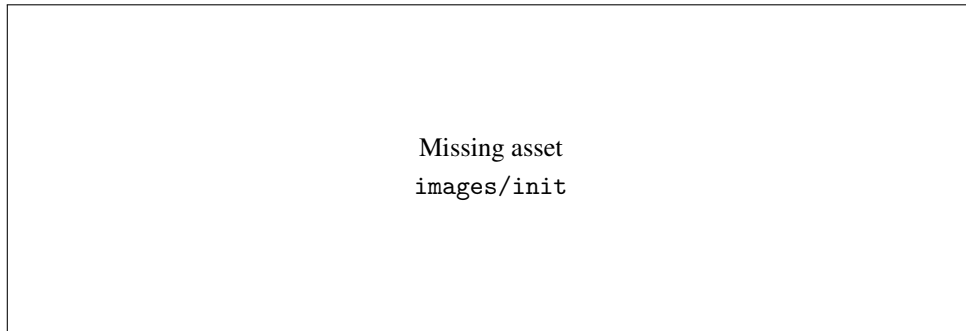


Figure 22: just use normal neural network inits, if you use zeros or ones no convergence. TODO still try the elser init method

537

538 C Hybrid architectures

539 To evaluate the framework’s interoperability with standard gradient-based modules, we explore a
 540 hybrid architectural design. Motivated by the current absence of a native projection-based embedding
 541 layer in $\mathcal{P}\text{Torch}$, we tackle the SST-2 text classification benchmark by combining optimization
 542 paradigms. Specifically, we construct a model utilizing a standard gradient-optimized embedding layer
 543 feeding into a projection-optimized Multilayer Perceptron (MLP) within $\mathcal{P}\text{Torch}$. For comparison,
 544 we evaluate a fully gradient-based equivalent in torch and an analogous architecture in pjax. This
 545 hybrid approach highlights the flexibility of $\mathcal{P}\text{Torch}$, demonstrating that researchers can seamlessly
 546 bridge projection-based and gradient-based training within a single computational graph.

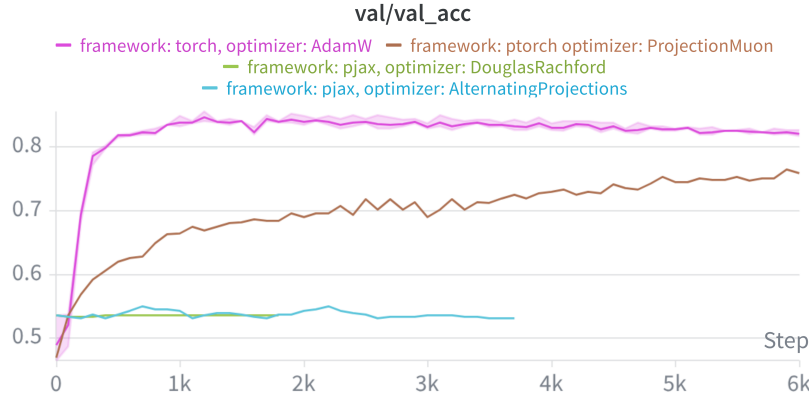


Figure 23: Performance on the STT2 benchmark -> regular torch wins $\mathcal{P}$ Torch better pjax not at all

D Comprehensive List of Projections

D.1 Layers

Linear Layer: Constraint

$$C_{\text{dot}} = \{(h, \theta, h^+) \mid \theta^\top h = h^+\}$$

Projection

$$\begin{aligned}\bar{h} &= \frac{h + t\theta}{1 - t^2}, \\ \bar{\theta} &= \frac{\theta + th}{1 - t^2}, \\ \bar{h}^+ &= h^+ - \frac{t}{\gamma^2}\end{aligned}$$

Derivation Source [bauschke_bilinear, 5, 2] *The scalar t is computed via Newton's method.*

deriving the weighted projection As section ?? builds up on a weighted version of this derivation, the full derivation is placed below for the linear layer constraint in l2 using lagrangian multipliers as in [5].

$$C_{\text{dot}} := \{(h, \theta, h^+) \in \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R} \mid \theta^\top h = h^+\}. \quad (26)$$

So we are searching for the closest point in the feasibility set under a weighted distance norm:

$$P_{C_{\text{dot}}}^{\alpha, \gamma}(h, \theta, h^+) \in \arg \min_{(\bar{h}, \bar{\theta}, \bar{h}^+) \in C_{\text{dot}}} \left(\|\bar{h} - h\|^2 + \alpha^2 \|\bar{\theta} - \theta\|^2 + \gamma^2 (\bar{h}^+ - h^+)^2 \right).$$

Given a current iterate (h, θ, h^+) and scaling parameters $\alpha, \gamma > 0$, the projection is

$$\begin{aligned} \min_{\bar{h}^+ \in \mathbb{R}, \bar{h}, \bar{\theta} \in \mathbb{R}^n} \quad & \|\bar{h} - h\|^2 + \alpha^2 \|\bar{\theta} - \theta\|^2 + \gamma^2 (\bar{h}^+ - h^+)^2 \\ \text{s.t.} \quad & \bar{h}^+ = \bar{\theta}^\top \bar{h}. \end{aligned} \quad (27)$$

Introduce a Lagrangian multiplier λ and define

$$\mathcal{L} = \|\bar{h} - h\|^2 + \alpha^2 \|\bar{\theta} - \theta\|^2 + \gamma^2 (\bar{h}^+ - h^+)^2 + \lambda (\bar{h}^+ - \bar{\theta}^\top \bar{h}).$$

Setting $\nabla \mathcal{L} = 0$ and using $t := \lambda/2$ yields

$$\bar{h} = h + t\bar{\theta}, \quad \bar{\theta} = \theta + \frac{t}{\alpha^2} \bar{h}, \quad \bar{h}^+ = h^+ - \frac{t}{\gamma^2}.$$

557 Solving the coupled system gives

$$\bar{h} = \frac{\alpha^2}{\alpha^2 - t^2}(h + t\theta), \quad \bar{\theta} = \frac{\alpha^2\theta}{\alpha^2 - t^2} + \frac{th}{\alpha^2 - t^2}.$$

558 Substituting into the constraint $\bar{h}^+ = \bar{\theta}^\top \bar{h}$ leads to a scalar equation $f(t) = 0$:

$$h^+ - \frac{t}{\gamma^2} = \frac{(\alpha^2\theta + th)^\top \alpha^2(h + t\theta)}{(\alpha^2 - t^2)^2} = \alpha^2 \frac{p(\alpha^2 + t^2) + qt}{(\alpha^2 - t^2)^2},$$

559 where $p := \theta^\top h$ and $q := \|h\|^2 + \alpha^2\|\theta\|^2$. The root can be found efficiently with Newton's method,
 560 after which $\bar{h}, \bar{\theta}, \bar{h}^+$ are reconstructed from the expressions above.

561 D.2 Activation functions

Gapped Step Activation: Constraint

$$S_{\text{step}_\delta} = \{(x, y) \mid (y = 1 \wedge x \geq \delta) \vee (y = 0 \wedge x \leq -\delta)\}$$

Projection

Project onto the closest feasible branch.

562

ReLU Activation: Constraint

$$S_{\text{ReLU}} = \{(x, y) \mid y = \max(0, x)\}$$

Projection

$$p_1 = (\min(x_0, 0), 0)$$

$$p_2 = (\max(0, \bar{x}), \max(0, \bar{x})), \quad \bar{x} = \frac{x_0 + y_0}{2}$$

Select $p \in \{p_1, p_2\}$ minimizing Euclidean distance.

563

564 D.3 Loss functions

Margin Loss: Constraint

$$S_{\text{margin}} = \{(x, y) \mid x \leq 0, y \geq 0\}$$

Projection

$$P_{C_{y,m}}(x_0) = \begin{cases} \min(x_0, 0), & y \leq 0 \\ \max(x_0, m), & y > 0 \end{cases}$$

565

566 Derivation Source [5][2]

Categorical Cross-Entropy: Objective

$$\text{CE}(x, y) = \log \sum_{j=1}^d e^{x_j} - \langle x, y \rangle$$

Update (Proximal)

$$x^* = x_0 + \lambda(y - \text{softmax}(x^*))$$

Solved as a proximal operator.

567

References

- [1] H. H. Bauschke and W. M. Moursi. *On the Douglas-Rachford algorithm for solving possibly inconsistent optimization problems*. 2021. arXiv: 2106.11547 [math.OC]. URL: <https://arxiv.org/abs/2106.11547>.
- [2] A. Bergmeister, M. K. Lal, S. Jegelka, and S. Sra. “A Projection-Based Framework for Gradient-Free and Parallel Learning”. In: *arXiv preprint arXiv:2506.05878* (2025). arXiv: 2506.05878 [cs.LG]. URL: <https://arxiv.org/abs/2506.05878>.
- [3] M. Á. Carreira-Perpiñán and W. Wang. “Distributed optimization of deeply nested systems”. In: *CoRR* abs/1212.5921 (2012). arXiv: 1212.5921. URL: <http://arxiv.org/abs/1212.5921>.
- [4] R. L. Dykstra. “An algorithm for restricted least squares regression”. In: *Journal of the American Statistical Association* 78.384 (1983), pp. 837–842.
- [5] V. Elser. “Learning Without Loss”. In: *CoRR* abs/1911.00493 (2019). arXiv: 1911.00493. URL: <http://arxiv.org/abs/1911.00493>.
- [6] V. Elser and M. K. Lal. “Learning with Boolean threshold functions”. In: *arXiv preprint arXiv:2602.17493* (2026).
- [7] T. Frerix, T. Möllenhoff, M. Möller, and D. Cremers. “Proximal Backpropagation”. In: *CoRR* abs/1706.04638 (2017). arXiv: 1706.04638. URL: <http://arxiv.org/abs/1706.04638>.
- [8] L. Gubin, B. T. Polyak, and E. Raik. “The method of projections for finding the common point of convex sets”. In: *USSR Computational Mathematics and Mathematical Physics* 7.6 (1967), pp. 1–24.
- [9] K. He, X. Zhang, S. Ren, and J. Sun. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *CoRR* abs/1502.01852 (2015). arXiv: 1502.01852. URL: <http://arxiv.org/abs/1502.01852>.
- [10] G. Hinton. “The forward-forward algorithm: Some preliminary investigations”. In: *arXiv preprint arXiv:2212.13345* 2.3 (2022), p. 5.
- [11] D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. “Difference Target Propagation”. In: *Machine Learning and Knowledge Discovery in Databases*. Ed. by A. Appice, P. P. Rodrigues, V. Santos Costa, C. Soares, J. Gama, and A. Jorge. Cham: Springer International Publishing, 2015, pp. 498–515. ISBN: 978-3-319-23528-8.
- [12] D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. “Difference target propagation”. In: *Joint european conference on machine learning and knowledge discovery in databases*. Springer, 2015, pp. 498–515.
- [13] S. B. Lindstrom and B. Sims. *Survey: Sixty Years of Douglas–Rachford*. 2020. arXiv: 1809.07181 [math.OC]. URL: <https://arxiv.org/abs/1809.07181>.
- [14] “Nonlinear programming”. In: *Journal of the Operational Research Society* 48.3 (1997), pp. 334–334.
- [15] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [16] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune. “Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning”. In: *arXiv preprint arXiv:1712.06567* (2017).
- [17] Y. Tang, Y. Tian, and D. Ha. “Evojax: Hardware-accelerated neuroevolution”. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 2022, pp. 308–311.
- [18] J. Wang, F. Yu, X. Chen, and L. Zhao. “ADMM for Efficient Deep Learning with Global Convergence”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’19. Anchorage, AK, USA: Association for Computing Machinery, 2019, pp. 111–119. ISBN: 9781450362016. DOI: 10.1145/3292500.3330936. URL: <https://doi.org/10.1145/3292500.3330936>.